

Livrable – E4



Franck Le Fur

Le 1^{er} avril 2026

Titre professionnel Développeur en intelligence artificielle
de niveau 6 enregistré au RNCP sous le n°37827.

Passage par la voie de la formation - parcours de 19 mois achevé en mars 2026



Table des matières

Présentation	4
Le Crédit Agricole	4
Le service ABP (Assurances des Biens et des Personnes).....	6
L'unité CAR.....	6
Organisation du projet [C14, C15, C16]	7
Construction d'un projet [C14].....	7
Cadre technique [C15]	8
Planification et suivi de la réalisation technique [C16]	8
Développement [C17]	10
Diagnostic des documents [C14]	10
Organisation du développement [C15]	10
Préparation des données.....	11
Chunking.....	12
Vectorisation des textes (TF-IDF, embedding).....	13
Recherche de Similarité Sémantique.....	14
FAISS (Facebook AI Similarity Search)	15
Interface Streamlit du moteur de recherche [C17]	16
Application Flask du moteur de recherche [C17, C18].....	17
Conception de l'application Flask [C17]	17
Système de Monitoring de l'application Flask.....	18
Test unitaire de l'application [C18]	18
Processus d'intégration continue [C19]	19
Conclusion	22
Aboutissement	22
Remerciements	23
Annexe 1 – Accompagnement des collègues [C14, C15].....	24
Test avec les équipes métier	24
Création de rôles pour l'application et ajout d'une mesure de sécurité.....	24
Actualisation journalière	24
Rédaction de documentations	25
Accompagnement des équipes et retours des collègues	26

Annexe 2 – Déploiement sur Serveur	26
Création d'un dossier ARM	26
Procédure d'installation de modules python en mode Offline	27
Préparer sur le PC d'origine un environnement complet avec tous les.....	27
Configuration du serveur	28
Automatisation des traitements	29
Création d'une tâche planifiée pour lancer les scripts	30
Authentification	30
Autorisations de l'application	30

Présentation

Le Crédit Agricole

Le Crédit Agricole est une banque française créée en 1894, c'est une banque **mutualiste**, ce qui signifie qu'elle est détenue par ses clients, appelés sociétaires, et que sa mission est de les servir plutôt que de maximiser ses profits pour des actionnaires externes.

En France, le Crédit Agricole est présent dans toutes les régions, avec un réseau de 3 200 agences et de 39 Caisses Régionales. Il est le leader de la banque de détail en France, avec environ 21 millions de clients particuliers et 900 000 clients professionnels et entreprises. Il est également présent dans d'autres secteurs financiers, tels que l'assurance, la gestion d'actifs et la banque d'investissement.

Le rôle principal du Crédit Agricole en France est de fournir des services financiers aux particuliers, aux professionnels et aux entreprises. Il propose une gamme complète de produits et de services financiers, tels que des comptes courants, des cartes bancaires, des crédits immobiliers, des prêts à la consommation, des assurances et des placements. Il se concentre également sur le financement de l'agriculture et de l'agroalimentaire, en raison de ses origines et de sa nature mutualiste.

Le Crédit Agricole est également impliqué dans de nombreuses initiatives sociales et environnementales en France. Il soutient des projets locaux et des associations en faveur de l'agriculture, de la culture, de l'éducation et de l'environnement. Il a également adopté une politique de développement durable, visant à réduire son empreinte environnementale et à promouvoir la finance responsable.

Le Crédit Agricole investit dans l'innovation technologique en France à travers plusieurs initiatives, notamment en créant des fonds de capital-risque pour financer des start-ups innovantes, en établissant des partenariats avec des incubateurs (**Le Village By CA à Ploufragan, Côtes d'Armor**) et des accélérateurs, et en développant des programmes internes pour encourager l'innovation et la transformation digitale.

En résumé, le Crédit Agricole est une banque mutualiste française qui fournit des services financiers aux particuliers, aux professionnels et aux entreprises. Il est le leader de **la banque de détail en France** et est présent dans d'autres secteurs financiers tels que **l'assurance et la banque d'investissement**. Il a également une forte implication sociale, environnementale et technologique en France.

La Caisse Régionale du Crédit Agricole des Côtes d'Armor (ou CR22) est engagée dans une politique de résultats durables pour garantir et pérenniser sa mission d'utilité économique et sociétale au service du territoire des Côtes d'Armor.

Bien au-delà de son rôle de banquier-assureur, **la CR22 (Caisse Régionale des Côtes d'Armor)** veut agir aux côtés des parties prenantes du territoire (chambres consulaires, collectivités locales, associations, entreprises, établissements de formation, autres Caisses Régionales du Crédit Agricole) pour le développement économique du territoire. Elle veut ainsi contribuer activement à la cohésion sociale et à l'attractivité des Côtes d'Armor.

A travers ses activités de collecte, de crédits, de gestion des flux monétaires et d'assurances, la CR22 irrigue l'économie et la création de richesse sur le territoire. Banque universelle, elle accompagne les projets de ses clients publics et privés sur la base d'une appréciation objective des risques.

Elle est à l'initiative ou elle collabore à la création ou au maintien de dispositifs visant à la création d'entreprises, à l'emploi, à la formation ou à l'innovation (Cafés de la création, Village by CA, Boutiques Starters, Tandem22, partenariat avec les établissements d'enseignement, Oscars des Entreprises, Trophées du Tourisme, etc...).

Créer des relations partenariales et de mécénat durables, bâties autour de valeurs communes : la Caisse Régionale soutient financièrement des actions et des projets culturels, sportifs, de solidarité et de valorisation du patrimoine par une politique de mécénat et de partenariats active et durable avec les associations. La CR22 veille à ce que son engagement ne se limite pas aux seuls apports financiers, mais qu'il vise une relation partenariale durable et propice au dialogue entre les parties. Elle sait renforcer ou maintenir son action solidaire dans les périodes difficiles (crise du Covid 19, ...)

Le Crédit Agricole n'agit pas que sur les Côtes d'Armor mais sur l'ensemble du territoire breton et a une présence bien ancrée en son espace (cf. **Erreur ! Source du renvoi**

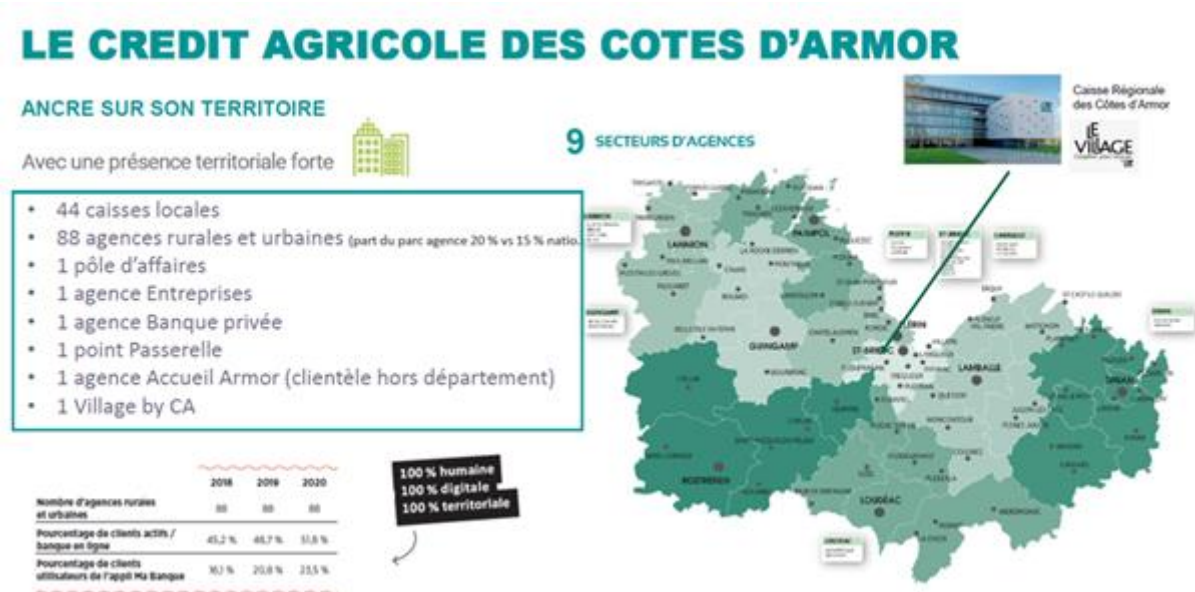


Figure 1 - Implémentation du Crédit Agricole sur le territoire breton

introuvable.).

Le service ABP (Assurances des Biens et des Personnes)

Le pôle **Assurances Biens et Personnes** est un pôle de conseil et d'accompagnement. Il a pour but de renseigner sur toutes les demandes touchant à l'assurance aussi bien personnelles (assurance vie, assurance santé, etc.) qu'aux assurances de biens (les assurances sur l'immobilier, les moyens de transport, etc.).

Le pôle ABP c'est 82 000 opérations d'assistance au cours de l'année, 280 000 actes de gestion, 38 000 alertes de surveillance et 48 000 opérations en lien avec les clients professionnels et agricoles.

Le pôle ABP est composé de 4 unités différentes, toutes ayant un champ d'action différent mais travaillant main dans la main les uns avec les autres :

- Conseil Assistance Réseaux (C.A.R)
- Unité Gestion Assurances (U.G.A)
- Unité Support et Projets (U.S.P)
- Agence en ligne Client Agriculteurs/Professionnels (A.C.A.P)

L'unité CAR

L'unité C.A.R a la charge du conseil et de l'accompagnement des conseillers en agence, c'est-à-dire qu'ils doivent répondre aux demandes d'assistance concernant la

souscription de contrats d'assurance, les modifications de contrats, les demandes de renseignements, les réclamations, les dérogations, les règlements d'impayés.

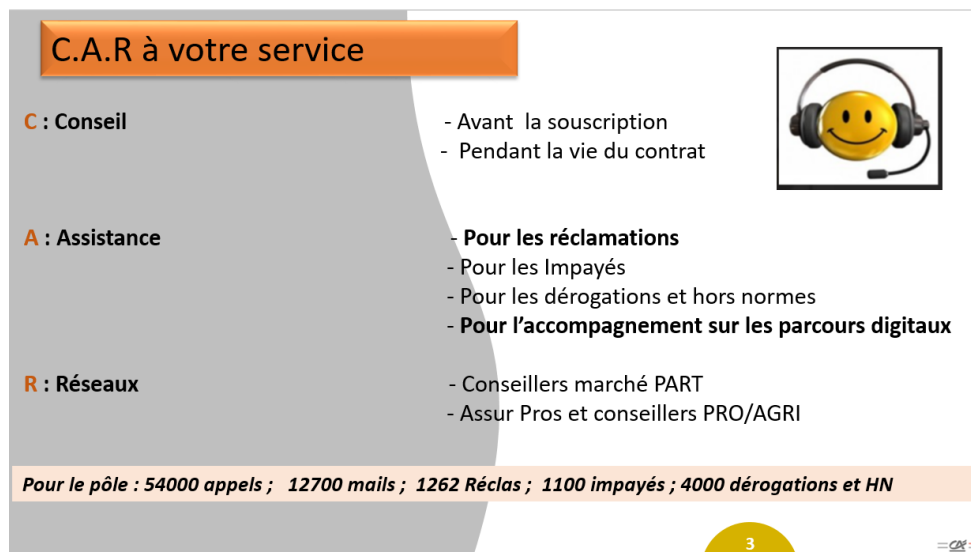


Figure 2 - Description des activités de l'unité C.A.R

Les demandes sont faites par téléphone ou bien rédigées à l'écrit via un outil appelé AVE, cet outil permet d'échanger et de suivre le parcours de la demande jusqu'à la résolution.

Organisation du projet [C14, C15, C16]

Construction d'un projet [C14]

Les premières semaines de mon alternance ont été consacrées à la recherche d'un projet incluant de l'IA, je suis donc allé dans les différentes unités pour entendre et comprendre leurs problématiques du quotidien, les premières observations sont que :

1. Leurs problématiques relèvent surtout de **problèmes d'automatisation de processus** mais pas d'IA,
2. Leurs outils sont développés par l'entité **CASIHA** au national et nous n'avons pas la main dessus au niveau régional pour y ajouter des développements,
3. Ils ne possèdent **pas de données exploitables** (par exemple leur outil principal de messagerie est développé au national et la base de données qui contient les messages ne nous est pas accessible).

Lors de différentes réunions avec mes responsables, je les ai sensibilisés sur le fait qu'**un projet d'IA devait se construire autour de la donnée**, lors de mes différentes immersions dans les 4 unités d'ABP j'ai été en tandem avec certains collègues, par exemple lors de leurs traitements des appels téléphonique (double écoute) et lors du traitement des demandes écrites via leur outil de messagerie.

Cela m'a permis d'observer **une problématique constante** :

Les collègues ne savent jamais où retrouver leurs documents, certains documents sont sur des répertoires partagés par le service, parfois sur des répertoires partagés par l'unité, parfois sur des outils de recherche ou parfois les documents sont sur leur ordinateur (téléchargés à partir d'un email etc ...).

Finalement un projet s'est dégagé des autres, il s'agit de « **la création d'un moteur de recherche contenant les procédures internes du service Assurance** ».

Ce projet aura comme bénéfices pour les collègues banquiers-assureurs :

- Centralisation de la connaissance ABP,
- Facilitation la recherche documentaire,
- Meilleur partage entre les unités.

Lors de différentes réunions, des demandes spécifiques seront ajoutées au projet, par exemple :

- La mise à jour automatique des documents dans le moteur de recherche après modification des documents,
- Aperçu des documents dans le moteur de recherche,
- Possibilité de rechercher dans le titre du document ou dans le contenu du document.

Cadre technique [C15]

Le projet est développé en langage **Python (3.12)** à l'aide de l'IDE **VS Code**, nous créons des environnements virtuels à l'aide de **VENV** et l'on privilégiera tant que possible la **programmation orientée objet**.

L'application qui sera livrée au Crédit Agricole est développée à l'aide du **framework Streamlit** qui permet de faire rapidement du développement WEB, une application équivalente développée à l'aide du **framework Flask** sera développée pour les besoins de la formation.

Les librairies python :

- **Numpy, Pandas, Scipy** pour la manipulation de DataFrame et de données numériques,
- **Scikit learn** et **Sentence-Transformers** pour les modèles de Machine Learning et pour les modèles d'embedding,
- **LangChain, FAISS, ChromaDB** pour les outils de recherche de similarité et le stockage de données vectorielles.

Le Crédit Agricole héberge tous ses développements sous **GitLab**, nous travaillons sur une branche « **Develop** » et lorsqu'une fonctionnalité nous semble opérationnelle nous faisons un « **Merge Request** » vers la branche « **Main** » et nous attribuons la tâche de **reviewer** à notre responsable de développement pour la validation.

Enfin nous disposerons en fin d'alternance d'un serveur Apache pour le déploiement et la mise à disposition de l'application finale à nos collègues des assurances.

Planification et suivi de la réalisation technique [C16]

Une étape de réflexion et de planification a permis de créer **un diagramme de Gantt**, permettant ainsi de mieux suivre l'avancement du projet avec les responsables.

Pour ce **diagramme de Gantt**, nous avons utilisé l'outil interne au Crédit Agricole appelé « **Gladys** » qui permet de créer et de suivre un projet en créant des tâches et en les assignant aux différents membres de l'équipe.

Planning des réunions :

- Réunion mensuelle auprès du responsable de service et des différents responsables d'unités devant qui je dois présenter mes avancées sur ce projet et éventuellement discuter de mes besoins ou des problèmes rencontrés.
- Réunion hebdomadaire avec les équipes informatiques (Pôle Péri breton) pour s'assurer que nous ne sommes pas bloqués sur des outils techniques.

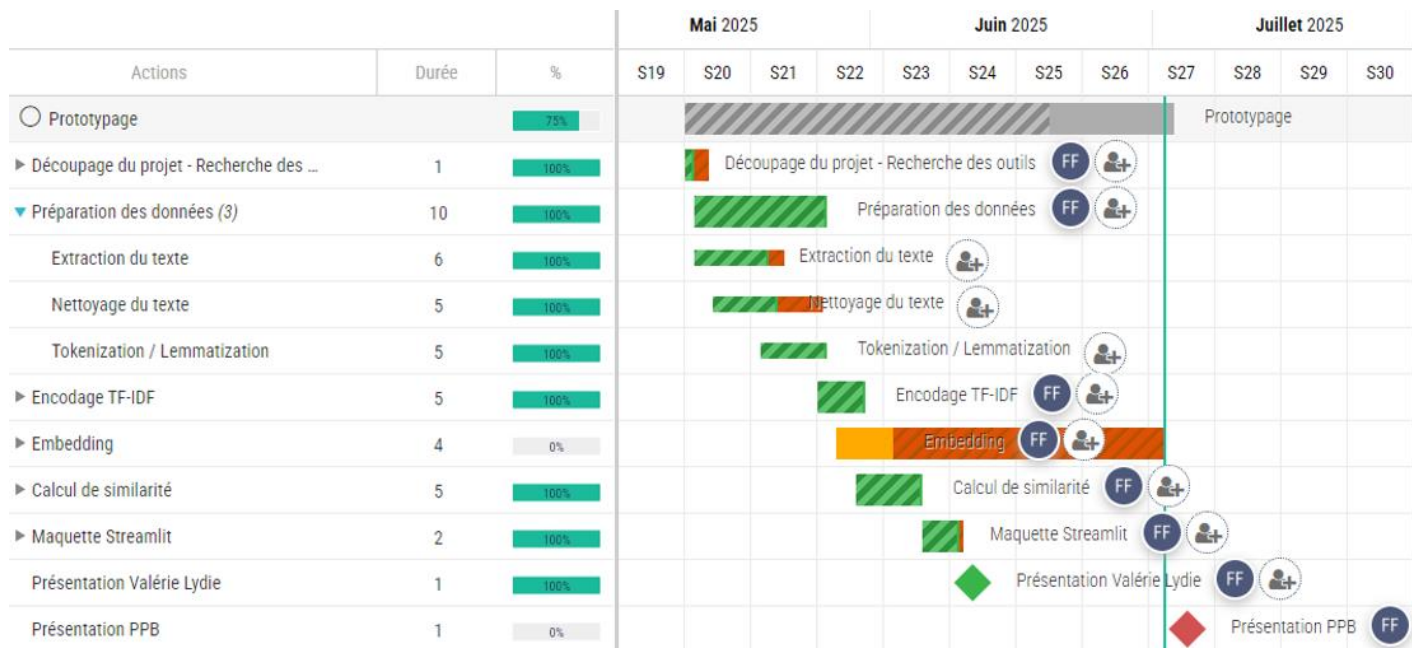


Figure 3 - Gantt pour la conception d'une maquette (POC)

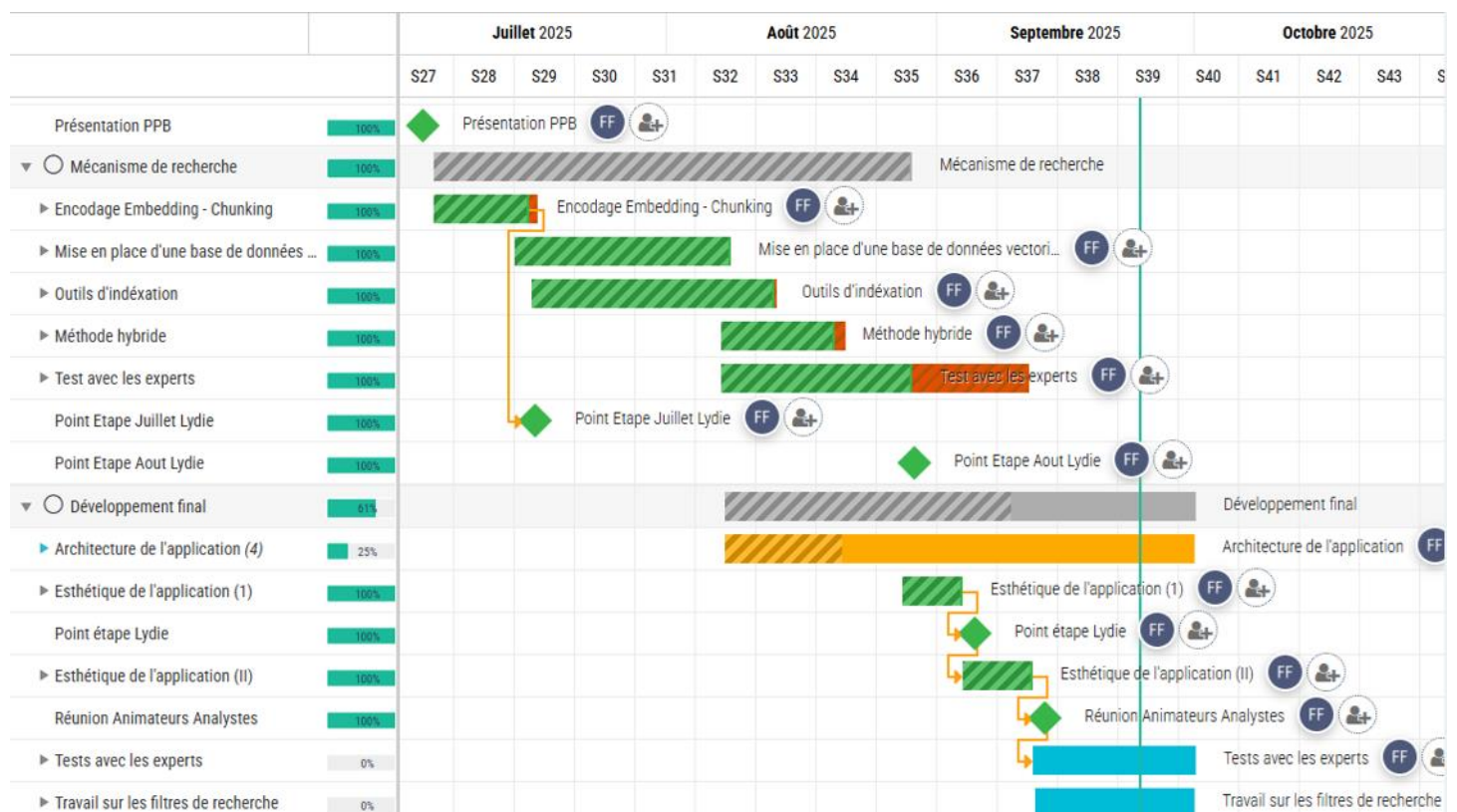


Figure 4 - Gantt pour le développement de l'application

Développement [C17]

Diagnostic des documents [C14]

Avant de se lancer dans le développement d'un moteur de recherche, le premier travail consiste à aller dans les 4 unités et comprendre quels documents doivent être pris en compte dans l'outil, finalement on compte environ 800 documents qui concerne principalement des documents de type :

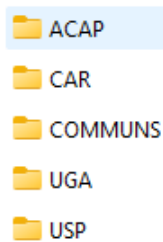
Procédures internes, Process internes, bagages techniques, supports de formation, supports pédagogiques, cartes des CAP, historiques des conditions générales.

Ces documents sont aux formats : PDF, DOCX, XLSX, PPTX, XLSM, DOCM, XLS.

Nous décidons donc d'une arborescence pour centraliser ces documents sur un répertoire dédié à cette bibliothèque ainsi que de certaines règles de nommage des fichiers et d'utilisation de l'outil « moteur de recherche ».

Première préconisation :

Distinction des arborescences des 4 unités



Avantages :

- Faciliter d'actualisation des documents,
- ne pas mélanger les documents de chaque unité,
- permet de conserver l'arborescence déjà existante par unité,
- si l'on renomme les unités : on renomme les dossiers OK
sinon dans le cas où l'on a mis un préfixe d'unité sur chaque fichier c'est beaucoup plus long à maintenir.

Deuxième préconisation : consigne de nommage des fichiers (exemple CG pour conditions générales et « Carte grise » en toute lettre, pas d'abréviations)

Organisation du développement [C15]

Une fois le diagnostic des documents établis, nous construisons un plan de développement d'une maquette **Streamlit** pour présenter aux équipes.

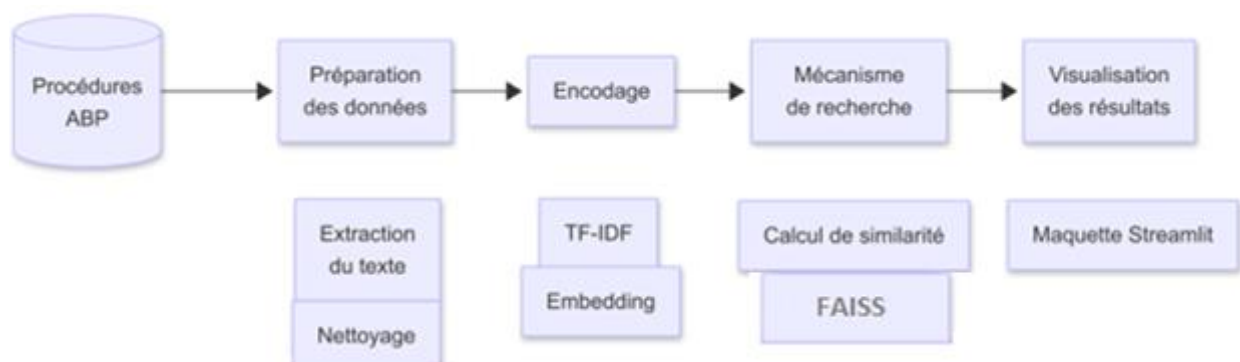


Figure 5 - Diagramme du flux de données

Préparation des données

Les **données textuelles** sont des **données non structurées** et nécessitent d'être préparées avant d'être utilisées dans un calcul de recherche de similarité.

Extraction des textes

La première étape est d'extraire le texte des documents qui sont de types PDF, PowerPoint, EXCEL et WORD (certains fichiers Word et Excel contiennent des macros), On extrait les textes des documents à l'aide des librairies python :

Pdfplumber, python-pptx, openpyxl, python-docx

Conversion des documents en PDF

Pour offrir un aperçu des documents dans le moteur de recherche, on convertit les fichiers PowerPoint, Word et Excel au format PDF, ce sont finalement ces PDF qui seront présentés dans l'application.

Nous testons différentes librairies python de conversion de fichiers, par exemple :

- La librairie **Spire** est une librairie très rapide et performante mais elle ajoute aux documents un Watermark d'avertissement d'évaluation du logiciel, pour la supprimer il faut acheter une licence,
- La librairie **Reportlab** qui est également très rapide, extrait le texte brut et le met en pdf, elle ne conserve ni la mise forme du texte ni la mise en page.

Finalement les librairies retenues pour la conversion en PDF sont :

Doc2pdf pour la conversion de WORD en PDF, **pptxtopdf**, pour la conversion de PowerPoint en PDF et la librairie **win32com** pour la conversion de EXCEL en PDF, cette dernière librairie prend également en charge les fichiers Excel contenant des macros.

Nettoyage des textes

Les textes extraits sont bruts et nécessitent d'être « nettoyés », c'est-à-dire formatés avant d'être vectorisés. Dans cette étape nous effectuons les traitements suivants :

- Passage en minuscules,
- Suppression des chemins vers des répertoires,
- Suppression des caractères spéciaux,
- Supprimer des suites de « XXX » et de « » présents dans certains fichiers,
- Supprimer les retours à la ligne et les espaces inutiles.

Tokenization et Lemmatization avec Spacy

Dans cette étape nous utilisons la librairie de **NLP Spacy**, cette librairie s'appuie sur un vocabulaire, dans notre cas nous utilisons le modèle français : « **fr_core_news_md** », ce modèle contient son propre ensemble de « Stop Words » c'est-à-dire de mots non

significatifs dans les textes (le, la, les, du, des, avoir, vers, je, elle, et, car etc...), cette liste de stop-words doit être ajustée pour notre problème, par exemple les mots « CAR » et « HORS » doivent être retirés de cette liste car ils sont très importants dans notre cas particulier de documents dans le contexte des assurance.

Une fois avoir chargé le modèle français de la librairie Spacy nous pouvons effectuer les étapes suivantes :

- **Tokenization** des textes,
- Suppression des **stop words, des nombres** et des marques de **ponctuation**,
- **Lemmatization** des textes,
- Finalement on termine ce traitement de texte en supprimant tous les accents à l'aide de la fonction **unidecode**.

Chunking

Cette étape consiste à découper les textes nettoyés en unités de taille homogène, en effet dans notre moteur de recherche lors de la recherche de similarités sémantiques, il est préférable de comparer la saisie utilisateur à des chunks de taille uniforme plutôt que de la comparer à des documents dont la taille peut varier entre 10 mots et 5000 mots. Un second intérêt est que les modèles d'embedding ont souvent une limite de taille d'entrée.

Différentes Techniques de chunking

- **Chunking fixe** : division selon une taille prédéfinie (nombre de caractères, mots ou tokens).
- **Chunking basé sur les phrases** : segmentation par frontières de phrases, garantissant des unités complètes.
- **Chunking thématique** : détection des changements de sujet ou de thème pour créer des segments cohérents.

Après différents tests nous avons choisi de faire un **chunking fixe** (256 tokens) avec un **chevauchement entre chunks** de 48 tokens (pour éviter la perte d'informations).

Le chunking se fait à l'aide de la librairie **LangChain**.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=Config.CHUNK_SIZE, # 256
    chunk_overlap=Config.CHUNK_OVERLAP, # 48
    length_function=len
)
```

Vectorisation des textes (TF-IDF, embedding)

Une fois les données préparées, les chunks doivent être **vectorisés** afin de pouvoir faire de la **recherche de similarité sémantique**, notamment à l'aide de calculs de type **similarité cosinus**.

Après études théoriques et tests pratiques des méthodes de vectorisation classiques (voir le livrable E5 – Veille technologique) nous avons retenu **2 techniques de vectorisation** en lien avec les **2 modes de recherche** qu'offre le moteur de recherche :

1. Une vectorisation **TF-IDF** pour **la recherche dans les titres des documents**,
2. Une vectorisation par **embedding** pour **la recherche dans le contenu des documents**.

TF-IDF

Représentation vectorielle des textes en fonction de l'importance statistique des mots dans le corpus de textes.

Embedding

L'**embedding** ou **plongement lexical** est une technique qui génère des représentations vectorielles denses capturant la dimension sémantique des mots / phrases.

Dans ce projet nous choisissons le modèle : **Sentence-CamemBERT-Large**, modèle pour la langue française développé par **La Javaness**.

Caractéristiques principales du modèle Sentence-Camembert-Large

- Le modèle est fine-tuné à partir du modèle **Facebook/camembert-large** en utilisant des réseaux Siamese BERT avec sentence-transformers sur le dataset **STS-B**.
- Performance : Il offre des capacités de **recherche sémantique puissantes**, tout est restant léger (337 millions de paramètres),
- Applications : Idéal pour des tâches telles que la recherche sémantique, le regroupement de texte, les systèmes de recommandation et la réponse aux questions,
- Dimension des vecteurs d'embedding : 768

Évaluation sur le dataset STS-B

Model	Pearson correlation	Spearman correlation
dangvantuan/sentence-camembert-large	85.9	85.8
dangvantuan/sentence-camembert-base	82.36	81.64

Source : <https://huggingface.co/dangvantuan/sentence-camembert-large>

(Pour plus d'informations sur les méthodes de Vectorisation :

Voir le **livrable sur la veille technologique E2)**

Recherche de Similarité Sémantique

Une fois nos documents vectorisés, nous pouvons collecter une phrase utilisateur, puis la nettoyer et la vectoriser **en utilisant les mêmes méthodes que le traitement précédemment fait pour nos documents** et effectuer une recherche de similarité sémantique, c'est-à-dire une comparaison du vecteur « utilisateur » par rapport à l'ensemble des vecteurs provenant de nos documents.

La similarité cosinus

La mesure couramment utilisée pour comparer des vecteurs est la « **similarité cosinus** », elle mesure la proximité entre deux vecteurs de l'espace en mesurant l'angle entre ces deux vecteurs : plus cet angle est faible, plus les vecteurs sont considérés comme similaires.

Une similarité cosinus élevée indique donc des textes sémantiquement proches.

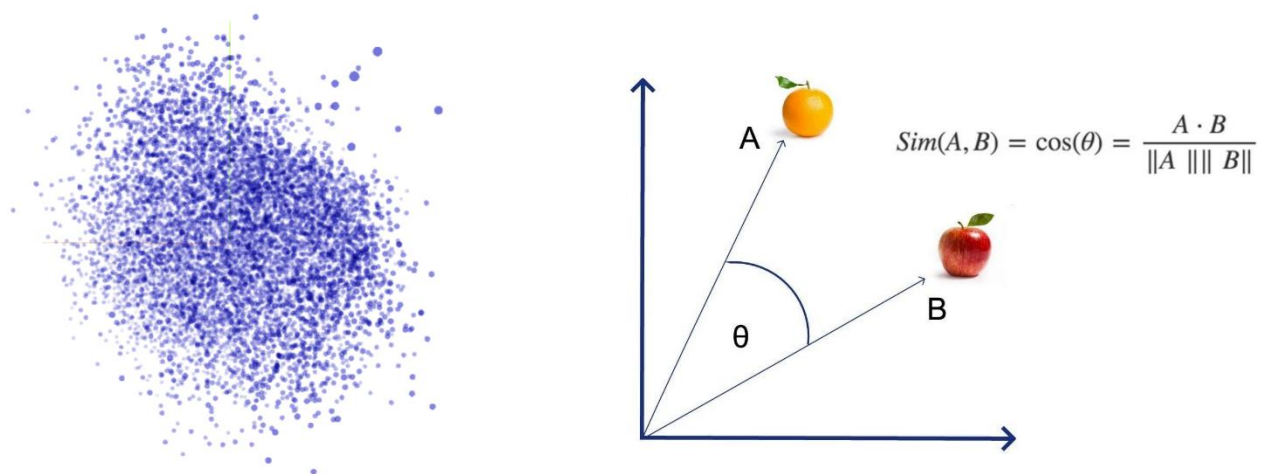


Figure 6 - Similarité Cosinus

Pour chaque requête utilisateur, nous calculons la similarité cosinus entre le saisie utilisateur et tous nos chunks.

1^{er} cas : vectorization **TF-IDF**

On réutilise le vectorizer utilisé pour notre corpus de documents,

```
tfidf_query = tfidf_vectorizer_titles.transform(query)
similarities = cosine_similarity(tfidf_query, tfidf_matrix_titles).flatten()
```

2^{ème} cas : vectorization par **embedding**

Les modèles de Sentence-transformers possède une méthode encode (pour réaliser l'embedding) ainsi qu'une méthode similarity (pour le calcul de similarité cosinus).

Voir lien : www.sbert.net/docs/

```
similarities = model.similarity(query_embedding, chunks_embedding)[0].numpy()
index_scores = similarities.argsort()
```

Index_score : vecteur de taille le nombre de chunks classés **par valeur de similarité cosinus dans l'ordre croissant**, c'est-à-dire par **pertinence inversée**.

On parcourt donc ce vecteur dans le sens inverse pour commencer par les chunks les plus pertinents, ensuite on utilise une table de correspondance entre les chunks et les documents, on récupère ainsi les 10 documents les plus pertinents.

```
top_k = min(10, len(documents))
counter = 0
indexes_result = []

for j in index_scores[::-1]:
    if mapping_chunks_documents[j] in indexes_result:
        continue

    print(f" - {titles[mapping_chunks_documents[j]]: <50}: {similarities[j]:.4f}")
    indexes_result.append(mapping_chunks_documents[j])
    if len(indexes_result) == top_k:
        break
```

Cette double approche (**TF-IDF et embedding**) permet de comparer les textes non seulement **sur la base des mots exacts utilisés**, mais aussi sur **leur sens global**.

Le moteur de recherche offre donc 2 modes de recherche différents, tous les 2 pertinents selon que l'utilisateur sache précisément ou non ce qu'il recherche.

FAISS (Facebook AI Similarity Search)

FAISS est une librairie open source développée par **META AI Research** pour faciliter la recherche de similarité sémantique dans de larges jeux de données.

L'un des principes (et astuce) de FAISS est le principe d'indexation par fichiers inversés, on regroupe les vecteurs similaires en clusters (par exemple avec l'algorithme des K-Means), ensuite pour une requête on commence par identifier les clusters les plus proches du vecteur requêtes (en faisant **un calcul de similarité entre la requête et les centroïdes des clusters**), ensuite on procède par calcul de similarité à l'intérieur de ces clusters. On évite ainsi de faire des calculs avec toute la base et on évite également de charger en mémoire toute la base.

Après avoir testé la librairie FAISS, il apparaît que dans notre cas d'usage au Crédit Agricole il n'est pas pertinent d'utiliser un tel outil avec si peu de documents.

Interface Streamlit du moteur de recherche [C17]

On développe l'interface utilisateur à l'aide du framework python **Streamlit**, framework open-source qui permet de créer facilement des applications web sans connaissances en développement WEB (contrairement à un framework comme **Django**).

On peut ajouter des **widgets interactifs** (curseurs, boutons, checkbox) et afficher des listes, des dataframe ou des graphiques avec des instructions python, Streamlit génère automatiquement le code html, il nous est ensuite possible d'ajouter du CSS pour personnaliser le rendu de l'application. Streamlit permet de prototyper et de déployer rapidement des outils interactifs, en se concentrant sur la logique métier plutôt que sur les détails techniques du web.

Cette application Streamlit est la version finale de l'application mise en production au Crédit Agricole et utilisée par le service **Assurance**.

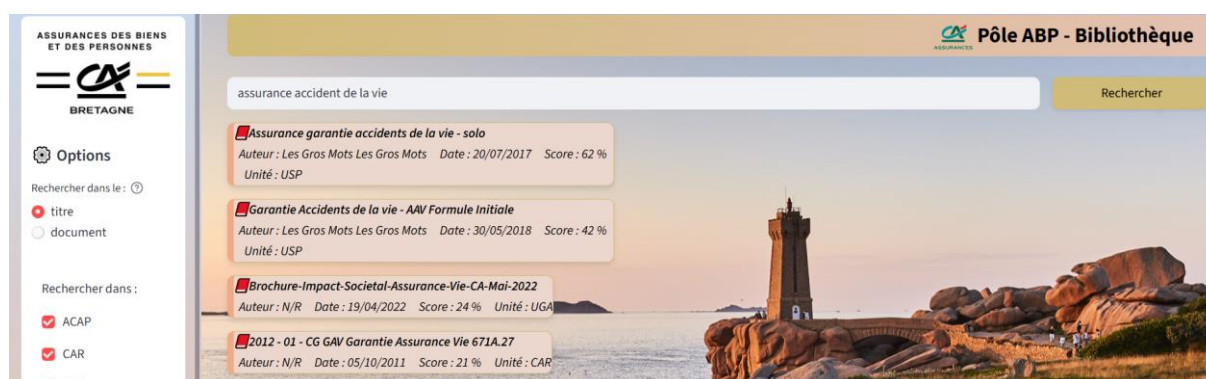


Figure 7 - Moteur de Recherche Streamlit

Ce projet s'est poursuivi avec des réunions avec les analystes de chaque unité pour effectuer des tests et des ajustements ainsi qu'une phase d'accompagnement des collègues assureurs (Voir [Annexe 1 - Accompagnement des collègues](#)).

Enfin l'application « moteur de recherche ABP » a été déployé sur un serveur Apache, pour les détails sur la mise en production du moteur de recherche sur serveur (voir [Annexe 2 - Déploiement sur Serveur](#)).

Application Flask du moteur de recherche [C17, C18]

Afin de pouvoir faire du monitoring de l'application et de mettre en place un processus d'intégration continue, j'ai développé un équivalent de ce moteur de recherche grâce à la librairie **Flask**, **ce projet est personnel et n'est pas déployé au Crédit Agricole**.

Conception de l'application Flask [C17]

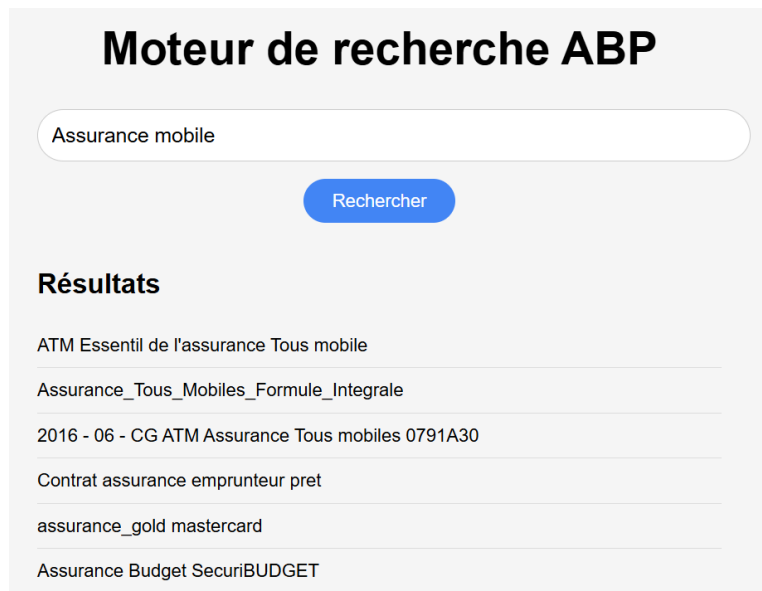
Flask est un micro framework open-source de développement web en python, le principe est d'utiliser des décorateurs (@app.route) pour associer des URLs à des fonctions Python (principe de routage), cela fait un pont entre interface Web et code Python, ce pont est assuré par le principe de **templating** (Jinja2 est le moteur de templates Flask) qui permet d'échanger des variables entre les pages html de l'application et le code python de l'application.

Exemple de templating Flask dans la ligne de code suivante :

```
render_template("index2.html", results=results)
```

On transmet l'objet python « results » à la page html appelée « index2.html ».

Nous créons une page html pour permettre à un utilisateur de saisir du texte et qui renverra une liste de résultats (comme dans un moteur de recherche classique).



Moteur de recherche ABP

Assurance mobile

Rechercher

Résultats

- ATM Essentiel de l'assurance Tous mobile
- Assurance_Tous_Mobiles_Formule_Integrale
- 2016 - 06 - CG ATM Assurance Tous mobiles 0791A30
- Contrat assurance emprunteur pret
- assurance_gold mastercard
- Assurance Budget SecuriBUDGET

Figure 8 - Moteur de recherche FLASK

Système de Monitoring de l'application Flask

On installe la librairie **Flask_Monitoring_Dashboard** qui permet de surveiller et d'analyser en temps réel les performances d'une application **Flask**.

Elle offre un suivi sur le suivi du nombre de requêtes, le temps de réponse par endpoint, mesurer le temps de réponse des endpoint, afficher l'origine des requêtes, identifier les routes lentes ou les requêtes problématiques ainsi que l'analyse des appels à la base de données.

On accède ensuite au tableau de bord via : <http://localhost:5000/dashboard>

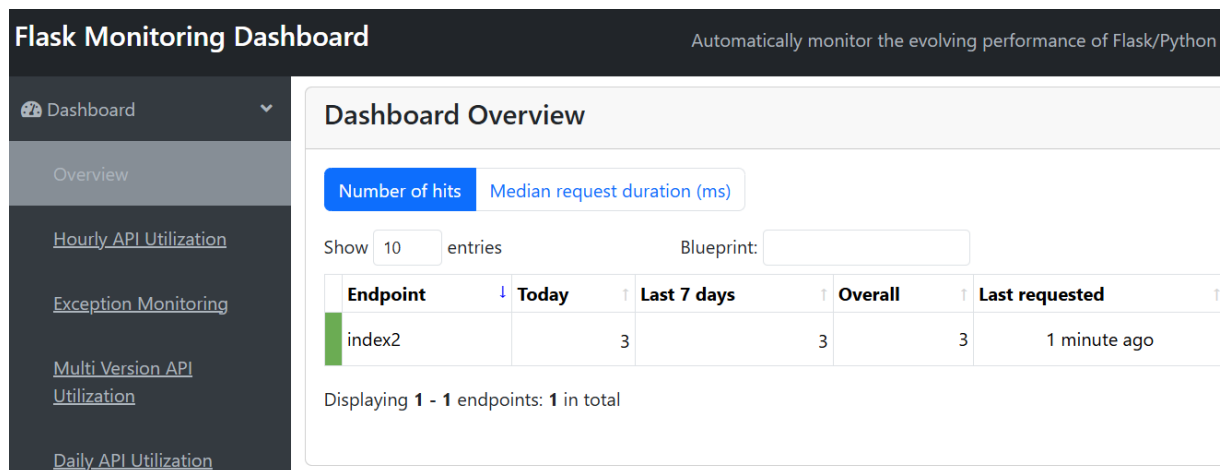


Figure 9 - Flask Monitor Dashboard

On peut ensuite sélectionner un endpoint (une route) pour obtenir des analyses détaillées, (volume d'utilisation du endpoint par heure de la journée, durée de réponse du endpoint), on peut également enregistrer des informations comme le login de l'utilisateur qui utilise l'application Flask, faire des regroupements sur certains critères pour par exemple observer l'utilisation de l'application par utilisateur).

Test unitaire de l'application [C18]

Nous utilisons la librairie **pytest** pour réaliser des tests unitaires sur notre application.

Exemple 1 : On vérifie que le fichier CSV contenant la liste des documents est bien présente et on vérifie que l'application parvient bien à charger le CSV.

Exemple 2 : Puisque nous allons faire des calculs de similarité cosinus, il est indispensable que les vecteurs (obtenus après vectorisation des textes) aient même dimension. On s'assure donc que la dimension du vecteur utilisateur (calculé à partir de la requête utilisateur) est égale à la dimension des vecteurs stockés dans la matrice TF-IDF.

Remarque : On peut lancer plusieurs fois le même test avec des arguments différents en utilisant le décorateur `@pytest.mark.parametrize`

```
@pytest.mark.parametrize("filename", ["tfidf_vectorizer_titles.pkl", "tfidf_vectorizer_texts.pkl"])
def test_load_vectorizer_titles(filename):
    print("\n***** Test Load Vectorizer")
    try:
        vectorizer_path = PATH_VEC + filename
        _ = Load_Vectorizer(vectorizer_path)
        assert True
    except:
        assert False, f"Impossible de charger le vectorizer depuis {vectorizer_path}."
```

Processus d'intégration continue [C19]

Pour l'instant notre application fonctionne en local en lançant la commande :
python app_flask.py, nous allons la déployer sur **Render**, Render est une plateforme cloud qui permet de déployer, exécuter et maintenir des applications web directement depuis un dépôt GitHub, avec déploiement automatique à chaque mise à jour du code.

Etapes du déploiement Render :

1. Création du fichier requirements.txt,

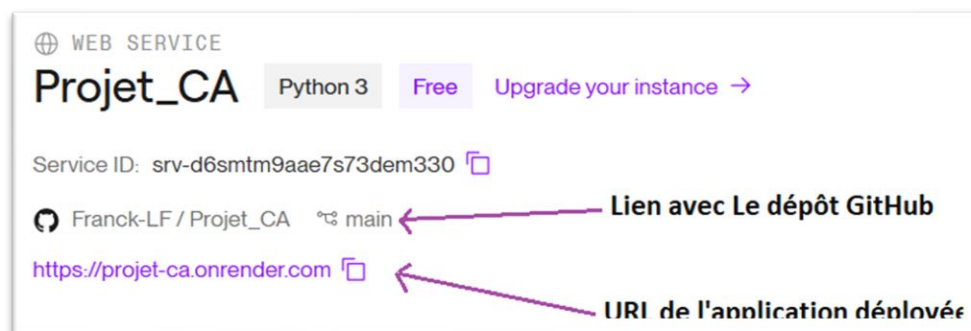
Render aura besoin de se construire un environnement identique au nôtre pour lancer notre application, nous transmettons à Render les versions des modules python à travers le fichiers "requirements.txt".

2. Dépôt de l'application sur Github,

On s'assure que le dépôt GitHub de l'application est mis à jour.

3. Création d'un webservice sur Render,

On relie le webservice à notre dépôt GitHub



4. Configuration du webservice pour automatiser le déploiement lors d'un commit

Build Command

Render runs this command to build your app before each deploy.

```
$ pip install -r requirements.txt
```

Commande à lancer avant déploiement pour construire l'environnement

Edit

Start Command

Render runs this command to start your app with each deploy.

```
$ gunicorn app_flask:app --bind 0.0.0.0:$PORT
```

Commande à exécuter lors d'un déploiement pour lancer notre application

Edit

Auto-Deploy

By default, Render automatically deploys your service whenever you update its code or configuration. Disable to handle deploys manually.

On Commit

▼

Pour automatiquement déployer l'application après chaque commit dans GitHub

Cancel

Save changes

Gunicorn est un serveur HTTP pour Python utilisé pour exécuter des applications Flask

On précise la version de python dans les variables d'environnements de Render :

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

Export ▼ Edit

KEY	VALUE
PYTHON_VERSION	3.12.10

5. Ajout d'instructions de déploiements dans le fichier main.yml

```

deploy:
  runs-on: ubuntu-latest
  needs: tests_job
  steps:
    - name: Deploy to production
      #uses: johnbeynon/render-deploy-action@v0.0.8
      uses: JorgeLNJunior/render-deploy@v1.4.6
      with:
        service_id: ${ secrets.MY_RENDER_SERVICE_ID }
        api_key: ${ secrets.MY_RENDER_API_KEY }
        wait_deploy: true

```

Ces instructions nécessitent de renseigner des secrets Render dans Github.

6. Ajout de « secrets » Render dans Github

Codespaces

Pages

Security

Advanced Security

Deploy keys

Secrets and variables

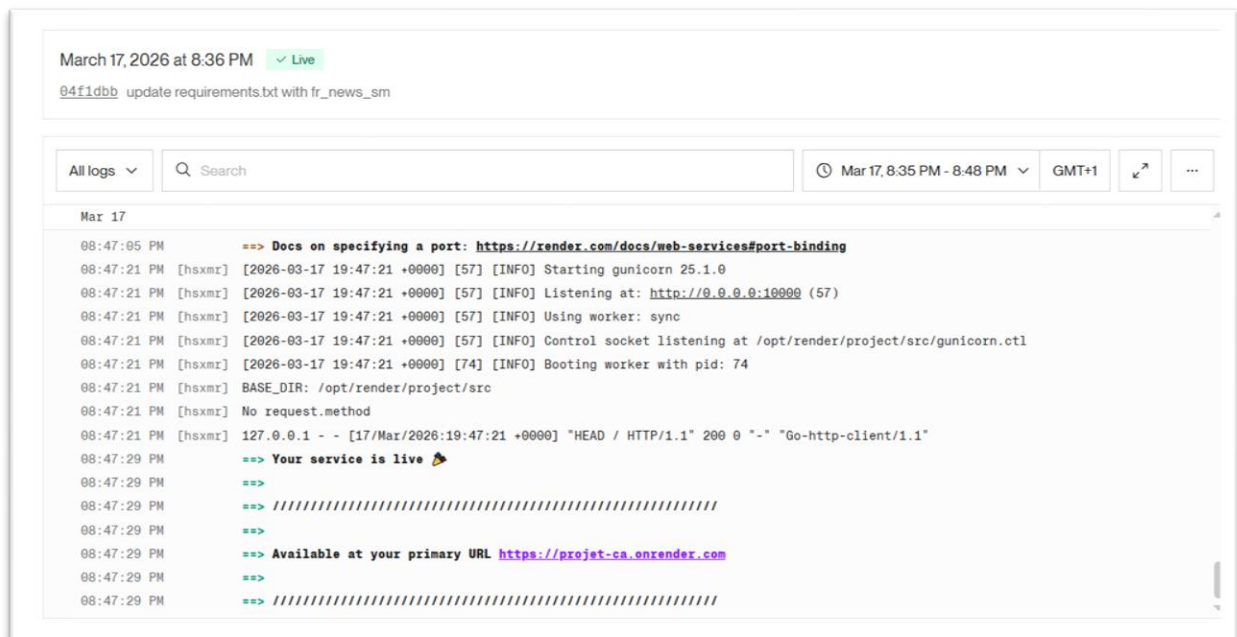
Actions

Repository secrets

New repository secret

Name ↕	Last updated
MY_RENDER_API_KEY	1 hour ago
MY_RENDER_SERVICE_ID	51 minutes ago

Le déploiement se fait dès validation des paramètres puis lors de chaque commit sur GitHub.



Suivi de l'état de nos différents déploiements :

Ungrouped Services

Active (1) Suspended (3) **All (4)**

Search services

SERVICE NAME	STATUS	RUNTIME	REGION	UPDATED
Binary_Classification	Suspended by you	Python 3	Oregon	50min
Projet_CA	✓ Deployed	Python 3	Oregon	14h
test	Suspended by you	Valkey 8	Oregon	1d
MLOps	Suspended by you	Python 3	Oregon	1y

L'application est désormais disponible à l'adresse : **<https://projet-ca.onrender.com/>**

Notons que nous utilisons une version gratuite de Render et que nous sommes limités dans le volume des librairies python du projet, pour cette raison nous avons supprimé certaines librairies gourmandes (MLFlow, ...) et nous utilisons une version SMALL de notre dictionnaire français pour Spacy.

Conclusion

Aboutissement

Le but de ce projet consistait à créer un moteur de recherche sur les documents internes au service ABP. L'objectif était de centraliser les connaissances du service, de permettre un meilleur échange des documents entre les 4 unités du service et de faire gagner du temps aux collègues avec un outil simple de recherche.

Le développement de l'application a respecté les objectifs fixés, de régulières présentations aux équipes métiers ont permis de recentrer les objectifs, notamment sur le choix des documents à incorporer dans l'outil.

La mise en production a été ralentie, notamment concernant les aspects : déploiement de l'application sur serveur, authentification sur le serveur, c'est avec l'aide de spécialistes que nous avons finalement pu configurer le serveur.

J'ai pu compléter ce projet de société avec des développements personnels (Application Flask, Monitoring, Déploiement) et ainsi balayer l'ensemble des compétences du référentiel de formation.

Ce projet m'a permis de travailler sur un sujet concret de société répondant à une problématique réelle, cela m'a amené à poser une réflexion sur l'organisation d'un projet complet, à rencontrer régulièrement les collègues des équipes métier. J'ai pu travailler sur le cycle complet de traitement de la donnée, de leur collecte, diagnostic, tri, nettoyage, traitement pour des tâches de NLP, vectorisation puis restitution de résultats (non pas sous forme de graphique mais sous forme de liste de résultats). J'ai réalisé la nécessité d'analyser longuement les données et de me référer aux équipes métier avant de se lancer tête baissée dans la programmation.

Pour aller plus loin dans ce projet, peut-être pour un futur alternant, on pourrait y intégrer un chatbot (RAG), permettant aux collègues de poser directement une question au moteur de recherche. Cette phase ne sera possible que lorsque des modèles d'IA générative seront disponibles au sein de la caisse régionale du Crédit Agricole et un projet de cette nature est déjà en cours de développement au Crédit Agricole mais au niveau national.

Cette fonctionnalité apporterait à la fois un gain de temps aux collègues dans la rédaction de leurs réponses mais aussi une uniformité dans les réponses du service.

Remerciements

Je souhaite exprimer toute ma gratitude à Sandrine, Marika, Lydie, Valérie et Gwenaël, pour leurs conseils précieux, leur accompagnement et la disponibilité qu'ils m'ont accordée sans compter. Le soutien que vous m'avez apporté lors de cette année a été déterminant pour la réussite de ce projet. Grâce à votre aide, j'ai pu approfondir mes connaissances en intelligence artificielle et développer des compétences essentielles pour mon parcours professionnel.

Je suis également très reconnaissant envers l'unité CAR qui par sa simple bonne humeur a rendu cette alternance très agréable. L'environnement de travail enjoué et l'accompagnement attentif ont été des facteurs clés de mon apprentissage et de mon épanouissement professionnel.

Je remercie également Yohann et Benoît du Pôle Péri Breton pour l'aide et le soutien qu'ils m'ont procurée au cours de l'année concernant l'accompagnement technique et sur les outils.

Votre contribution à tous a été essentielle pour mon développement professionnel et personnel.

Annexe 1 – Accompagnement des collègues [C14, C15]

Test avec les équipes métier

Une fois la 1^{ère} version de l'application terminée, nous planifions des rendez-vous avec les 4 analystes des 4 unités ABP pour réaliser des tests, discuter de l'application et prendre en compte leurs nouvelles demandes.

Les premiers tests avec les analystes sont concluants, à la fois sur l'ergonomie et la prise en main de l'outil que sur la pertinence des résultats du moteur de recherche. Les experts apprécient les différents modes de recherche et le fait de pouvoir saisir aussi bien des requêtes textuelles, numériques que mixtes (comme des codes produits A321GF, NH2020 ...), ils apprécient également l'insensibilité du moteur de recherche à la casse, au singulier / pluriel ou aux accents.

Lors de ces tests, certaines demandes apparaissent au niveau cosmétique ou bien fonctionnalité :

- Afficher l'auteur du document et la date de dernière modification,
- Filtre par unité (avec des CheckBox),
- Une mise à jour automatique du moteur de recherche lorsque des documents sont modifiés,
- Une fonctionnalité pour pouvoir retirer en urgence un document du moteur de recherche.

Création de rôles pour l'application et ajout d'une mesure de sécurité

Tous les collègues assureurs auront le rôle **utilisateur** sur le moteur de recherche avec les autorisations de consultations et de téléchargement des documents.

Il m'a été demandé d'ajouter un rôle de gestionnaire pour les responsables d'unités et animateurs, analystes de chaque unité, les gestionnaires auront la possibilité supplémentaire de retirer en urgence un document du moteur de recherche.

Cet ajout a été demandé par l'expert Sécurité Logique en charge de valider notre ARM, dans le cas (extrême) où un document contenant des données sensibles seraient déposé dans le répertoire dédié et deviendrait donc accessible aux collègues assureurs via le moteur de recherche, la fonctionnalité permet de retirer immédiatement le document sans attendre la mise à jour nocturne.

Actualisation journalière

L'application doit être dynamique et autonome c'est-à-dire qu'il faut permettre aux collègues de pouvoir ajouter de nouveaux documents, de les modifier ou de les

supprimer et que les modifications soient automatiquement prises en compte et le moteur de recherche soit automatiquement mis à jour.

Une tâche planifiée sera mise en place toutes les nuits sur le serveur pour analyser les documents, cependant sur les 800 documents présents, seul un faible nombre seront modifiés tous les jours il ne faut donc pas refaire tout le processus d'analyse de texte.

Pour faire cela, je mets en place un système qui reconnaît si un document a été ajouté ou modifié, on utilise pour cela une fonction de hashage (sha256) qui a la particularité de générer une clé unique par document.

Lorsque les documents sont scannés toutes les nuits, nous pouvons identifier les documents modifiés ou ajoutés et faire l'analyse de texte sur ces seuls documents. Le traitement nocturne passe de 3heures à quelques minutes.

Cette tâche automatique d'analyse des documents sera mise en place toutes les nuits à minuit. Les collègues seront informés que toute modification faite sur un document sera prise en compte dans l'application à J+1.

Rédaction de documentations

Rédaction d'une charte d'usage

Charte pour l'ensemble des collègues des assurances (85 personnes) pour spécifier :

- Le rôle de l'application,
- La finalité du moteur de recherche,
- Le public concerné,
- Les types et formats des documents concernés.

Rédaction d'une documentation technique

Destinée aux personnes autorisées à ajouter/supprimer/modifier des documents pour leur spécifier :

- Où déposer de nouveaux documents,
- La règle sur les types et formats de documents à déposer,
- La règle sur la conformité des documents à déposer (anonymisation),
- Action à mener pour retirer d'urgence un document.

Rédaction d'une documentation de maintenance technique

Documentation à l'usage du centre informatique du Crédit Agricole pour assurer la maintenance de l'application une fois que l'alternance sera terminée. Cette documentation détaille la technologie utilisée, les modules python nécessaires, décrit comment réinstaller l'environnement virtuel sur le serveur, les paramètres de configurations, comment relancer ou reconfigurer les tâches planifiées et enfin comment utiliser le moteur de recherche en lui-même.

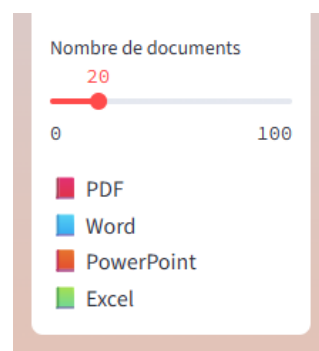
Accompagnement des équipes et retours des collègues

Plusieurs démonstrations sont faites pour présenter le moteur de recherche, une phase progressive de mise en production est faite pour les 4 unités.

La mise en production de l'application auprès des équipes métier a amené un certain nombre de retours et remarques.

Notons quelques problèmes d'affichage qui se sont réglés après avoir supprimés les cookies du navigateur et après avoir passé certaines chaînes de caractères en « **raw** » dans le code python.

Une autre remarque sur la nécessité de faire apparaître une légende concernant le type de document.



Un grand nombre de retours sur les documents que l'on ne retrouve pas dans l'application :

- Après analyse il s'avère que les documents ne sont pas présents dans le répertoire de notre bibliothèque. En discutant avec animateurs et analystes pour savoir si lesdits documents entrent bien dans le cadre de notre application nous les avons ajoutés dans le répertoire dédié, une fois fait les documents ont été ré-analysés automatiquement lors de la tâche automatisée la nuit et sont ensuite présents dans la liste des résultats de l'application.
- Certains documents sont bien présents dans le répertoire mais ne ressortent pas dans la liste des résultats, après analyse on s'aperçoit que les collègues ne font pas la recherche avec les bons mots clés ou bien qu'un travail de renommage des fichiers s'impose.

Annexe 2 – Déploiement sur Serveur

Création d'un dossier ARM

Avant de pouvoir travailler sur serveur pour déployer le moteur de recherche, nous devons demander les autorisations pour accéder au serveur à notre filiale (CAGIP) qui gère toutes les infrastructures de serveurs du Crédit Agricole.

Pour cela nous devons créer un dossier d'ARM (analyse de risques métier) pour évaluer les risques sur 3 aspects :

- RGPD

- Juridique et conformité
- Informatique (Sécurité Logique)

Une fois notre ARM validée nous sommes désormais autorisés à accéder à un serveur du Crédit Agricole, il s'agit d'un serveur Apache installé sur un Windows Server 2014.

Procédure d'installation de modules python en mode Offline

Le serveur ne peut pas aller sur internet, nous ne pouvons donc pas installer nos modules python avec des commandes telles que `pip install`.

Préparer sur le PC d'origine un environnement complet avec tous les modules python nécessaires.

- Faire un fichier requirements
`pip freeze > requirements.txt`
- Retirer la ligne avec le modèle Spacy :
`-e c:\users\s589669\documents\dev\test_env\fr_core_news_md-3.8.0`
- Télécharger les fichiers au format « whl » :
`pip download -r requirements.txt -d packages_offline`
Cela crée un répertoire “packages_offline” contenant tous les modules python.

Sur un autre PC (ou sur un serveur), créer un répertoire de travail

- Créer un environnement
`python -m venv env_server`
`env_server\Scripts\activate`
- Copier le fichier “requirements.txt” ainsi que le répertoire “packages_offline” dans le répertoire de travail
- Installer les modules à partir du répertoire “packages_offline”
`pip install --no-index --find-links=packages_offline -r requirements.txt`

`--no-index` → interdit à pip d'utiliser Internet

`--find-links=packages_offline` → pip cherche les fichiers uniquement dans ce dossier

Installation du modèle Spacy

Recopier le répertoire “fr_core_news_md-3.8.0”,

Pip install ne fonctionne pas sur le serveur mais lors du chargement du modèle on précise le sous-répertoire où se trouvent les fichiers meta.json et config.cfg

Par exemple :

```
self.nlp = spacy.load("E:\YBP10\ApachePub_Php8\PPB\bibliotheque-ABP\fr_core_news_md-3.8.0\fr_core_news_md\fr_core_news_md-3.8.0\")
```

Configuration du serveur

Nous disposons d'un serveur physique sur lequel nous devons configurer des « Virtual Host », c'est-à-dire des **serveurs virtuels** pour compartimenter le serveur physique.

```
<VirtualHost 10.70.107.19:8443>
  ServerName ybp10-iucrdevapa2.ca-technologies.fr:8443
  ServerAlias YBP10-IUCR.CW01PYBDP201.ca-technologies.fr:8443
  SSLEngine on
  SSLCertificateFile "D:\YBP10\Procparm\IUCR\APA\ssl\ybp10-iucrdevapa2.ca-technologies.fr\ybp10-iucrdevapa2.ca-technologies.fr.crt"
  SSLCertificateChainFile "D:\YBP10\Procparm\IUCR\APA\ssl\ybp10-iucrdevapa2.ca-technologies.fr\ybp10-iucrdevapa2.ca-technologies.fr-chain.pem"
  SSLCertificateKeyFile "D:\YBP10\Procparm\IUCR\APA\ssl\ybp10-iucrdevapa2.ca-technologies.fr\ybp10-iucrdevapa2_rsa_priv.key"
  DocumentRoot "E:/YBP10/ApachePub_Php8/PPB/"
  <Directory />
    Options -Indexes +FollowSymLinks
    AllowOverride All
    Require all granted
  </Directory>

  Define LOCAL_URL 127.0.0.1
  ProxyPreserveHost On
  ProxyPass /STC_Data_Analyst/ http://\${LOCAL\_URL}:84/STC\_Data\_Analyst
  ProxyPassReverse /STC_Data_Analyst/ http://\${LOCAL\_URL}:84/STC\_Data\_Analyst
  ProxyPass /bibliotheque-ABP/ http://\${LOCAL\_URL}:85/bibliotheque-ABP/
  ProxyPassReverse /bibliotheque-ABP/ http://\${LOCAL\_URL}:85/bibliotheque-ABP/

  <Proxy>
    Require all granted
  </Proxy>

  RewriteEngine On
  RewriteCond %{HTTP:Upgrade} =websocket
  RewriteRule /STC_Data_Analyst/(.*) ws://localhost:84/$1 [P]
  RewriteCond %{HTTP:Upgrade} !=websocket
  RewriteRule /STC_Data_Analyst/(.*) http://localhost:84/$1 [P]
  RewriteRule /bibliotheque-ABP/(.*) ws://localhost:85/$1 [P]
  RewriteCond %{HTTP:Upgrade} !=websocket
  RewriteRule /bibliotheque-ABP/(.*) http://localhost:85/$1 [P]
</VirtualHost>
```

Par exemple ici, on indique au serveur que toute requête URL de la forme :
`/bibliotheque-ABP/` doit être redirigée vers l'application qui tourne en localhost sur le port 85 (c'est-à-dire le moteur de recherche).

Automatisation des traitements

Nous écrivons deux scripts PowerShell : Un premier script pour faire l'analyse de documents (analyse textuel) et un second script pour lancer l'application streamlit.

Exemple du script pour le traitement nocturne des données :

```
$project_folder = "E:\YBP10\ApachePub_Php8\PPB\bibliotheque-ABP\"
$script_py = "traitement_nocturne.py"

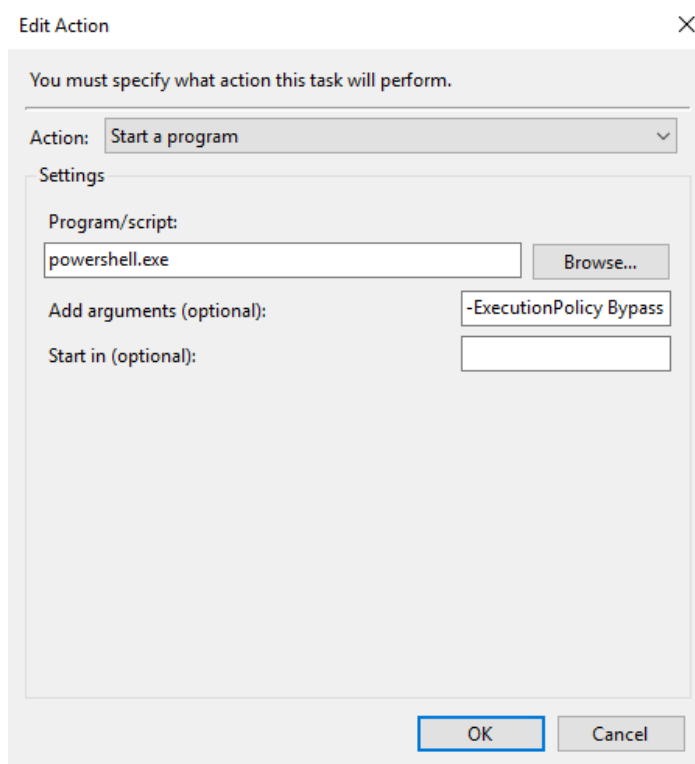
# Check if the folder with the project exists
if (Test-Path $project_folder)
{
    # Move to project folder
    Write-Host "Changement de répertoire " -ForegroundColor Green
    Write-Host $project_folder -ForegroundColor Yellow
    Set-Location $project_folder

    # Check if virtuel env exists
    if (Test-Path $virtual_env)
    {
        # Activate virtuel env
        Write-Host "Activation de l'environnement virtuel " -ForegroundColor Green
        Write-Host $virtual_env -ForegroundColor Yellow
        & $virtual_env

        # Check if python script exists
        if (Test-Path $project_folder$script_py) {
            & python$project_folder$script_py
        }
        else {
            Write-Host " ERROR - Script python introuvable" -ForegroundColor Red
            Write-Host $script_py - ForegroundColor Yellow
            exit 1
        }
    }
    else {
        Write-Host " ERROR - Environnement virtuel introuvable" -ForegroundColor Red
        Write-Host $virtual_env - ForegroundColor Yellow
        exit 1
    }

    # If application stops
    Write-Host " Arrêt de l'application" -ForegroundColor Green
}
else # No project folder
{
    Write-Host "ERROR - Répertoire introuvable ." -ForegroundColor Red
    Write-Host $project_folder -ForegroundColor Yellow
}
```

Création d'une tâche planifiée pour lancer les scripts



```
powershell.exe  
-ExecutionPolicy Bypass -NonInteractive -WindowStyle Hidden -File  
E:\YBP10\ApachePub_Php8\PPB\bibliotheque-ABP\script_app_streamlit.ps1
```

Authentification

Le Crédit Agricole dispose d'outils pour la sécurisation des applications, dans notre cas nous utilisons une mire d'authentification (développement interne au Crédit Agricole) qui utilise le protocole NTLM de Microsoft, ainsi les utilisateurs du Crédit Agricole n'ont qu'à s'identifier le matin sur leur poste et les applications internes ne redemandent pas d'authentification. En cas de déconnexion prolongée dans la journée, les utilisateurs se voient demander de se réauthentifier via la mire d'authentification afin de pouvoir réaccéder aux applications du Crédit Agricole.

Pour se faire il nous suffit d'enregistrer notre application dans le registre des applications du Crédit Agricole.

Autorisations de l'application

Nous enregistrons l'application dans le registre des applications du Crédit Agricole, il s'agit d'une application interne recensant toutes les applications.

Nous définissons les différents rôles possibles pour l'application :

	Niveau	Identifiant du profil	Libellé	Description	Nb droits appliqué(s)
	50	USER	USER	Droits de consultation des documents et de téléchargement des documents	4
	10	GESTIONNAIRE	GESTIONNAIRE	Droits de consultation des documents, de téléchargement des documents. Droits de retirer un index de l'outil (sécurité d'urgence pour retirer un document).	5
	0	ADMINISTRATEUR	ADMINISTRATEUR	Droits de consultation des documents, de téléchargement des documents. Droits de retirer un index de l'outil (sécurité d'urgence pour retirer un document).	1

Ensuite nous associons pour chaque rôle, les personnes ayant accès à l'application avec ce rôle et les droits associés, par exemple pour le rôle USER :

DROITS SUR LE PROFIL: USER											
Ajouter un droit											
		Type d'ajout	Valeur d'ajout	Valeur (Libellé)	Paramètre optionnel	Exclusion	Début	Fin	Description	Edité par	Date édition
		EDS	02076	ABP-UNITE GESTION ASSURANCES			26/01/2026			LE FUR Franck	26/01/2026
		EDS	02077	AGENCE CONSEIL ASSURANCES PROS			26/01/2026			LE FUR Franck	26/01/2026
		EDS	02154	ABP-UNITE SUPPORT ET PROJETS			26/01/2026			LE FUR Franck	26/01/2026
		EDS	02158	ABP-CONSEIL ASSISTANCE RESEAU			26/01/2026			LE FUR Franck	26/01/2026

Une fois les rôles et les personnes enregistrées dans cet outil, **un système de requêtage par API existe** pour connaître les droits d'un employé sur l'application en fonction de son numéro de matricule.

[https://p1ca-s4148-habilitationservice.prdo3.caas.ca-ts.group.gca/CRIF/auth_service/public/ws/getprofiluser?matricule=\[MATRICULE\]&application_id=\[APP_ID\]&version=1](https://p1ca-s4148-habilitationservice.prdo3.caas.ca-ts.group.gca/CRIF/auth_service/public/ws/getprofiluser?matricule=[MATRICULE]&application_id=[APP_ID]&version=1)

Le résultat est un fichier XML contenant les différents rôles de l'employé.

```
<response matricule="S589669" application="Bibliotheque_ABP">
  <status>1</status>
  <profil>
    <id_profil>ADMINISTRATEUR_IP</id_profil>
    <parametre/>
    <libelle>ADMINISTRATEUR_IP</libelle>
    <description>Droits de consultation des documents, de téléchargement des documents.
Droits de retirer un index de l'outil (sécurité d'urgence pour retirer un document).</description>
    <niveau>0</niveau>
  </profil>
  <profil>
    <id_profil>USER</id_profil>
    <parametre/>
    <libelle>USER</libelle>
    <description>Droits de consultation des documents, de téléchargement des
documents.</description>
    <niveau>50</niveau>
  </profil>
</response>
```